```
{"sku":"A","in_stock":true,"quantity":12}
```

The second version can be validated, displayed in a table, passed to another tool or stored in a database without guessing what the sentence meant.

Use JSON Schema or an equivalent schema system to describe important inputs and outputs. A schema might state that `quantity` must be an integer, `sku` must be a string and `in_stock` must be Boolean. Then validate returned data before the next step.

For a small Python project, install `jsonschema` and create a schema:

```
PRODUCT_SCHEMA = {
    "type": "object",
    "properties": {
        "sku": {"type": "string"},
         "quantity": {"type": "integer",
"minimum": 0},
        "in_stock": {"type": "boolean"}
    },
     "required": ["sku", "quantity", "in_
stock"],
    "additionalProperties": False
}
```

After a tool returns a dictionary, run `jsonschema.validate(result, PRODUCT_SCHEMA)`. If validation fails, handle the error rather than forwarding corrupt data.

Structured output does not make the model truthful. It only makes its output structurally predictable. "quantity": 5000 can be valid JSON and still be factually wrong. This is why authoritative values should come from deterministic tools where possible.

Also separate arguments proposed by the model from values supplied by your trusted application. If a tool requires a logged-in user ID, do not ask the model to invent that ID. The application should inject it from the authenticated session.

Build three useful local tools

Now replace the toy calculator with functions that make our local assistant more capable.

Tool 1: Search a SQLite catalogue

Create a small SQLite database containing product SKU, name, price and stock. Expose a read-only function:

```
def lookup_product(sku: str) -> dict:
    """Return product information for one
exact SKU."""
    ...
```

Use a parameterised SQL query such as `SELECT sku,name,price,stock FROM products WHERE sku=?`. Never build SQL by concatenating model-generated text. Return only columns the assistant genuinely needs.

Test three conditions manually: a valid SKU, an unknown SKU and a malicious-looking string. The third should remain a harmless parameter, not become part of the SQL command.

Tool 2: Summarise a CSV safely

A `summarise_csv(filename)` tool can load approved files using pandas and return column names, row count, missing-value counts and basic numeric statistics. But restrict it to `data/approved_csv`. Resolve the requested path and verify that it remains inside that directory. Reject `../../some/private/file`.

Do not dump a 100MB spreadsheet into the model's context. Tools should transform raw systems into the minimum useful result. Return, say, thirty summary values rather than every row.

Tool 3: Search your document index

Chapter 3 already gave us retrieval. Wrap that functionality as `search_documents(query, top_k=5)`. Return each passage with its source ID and page metadata. Now your assistant can decide when a question requires private documents rather than forcing retrieval on every message.

Existing capabilities can become tools. You are not rebuilding RAG; you are giving an orchestrator controlled access to it.

Test every function outside the model first. If `lookup_product("LP-101")` fails in ordinary Python, adding an LLM will not fix it. Write unit tests covering valid input, missing data, invalid types and permission failures. The model should be the last component you introduce, not the first.

Let the model choose between tools

Once individual tools work, expose several together. Suppose the user asks:

"Compare our stock for LP-101 with the minimum level mentioned in the replenishment policy."

One piece of information lives in SQLite; another lives in your indexed documents. The model may need `lookup_product` and `search_documents`.

The workflow becomes:

1. Send the request plus tool definitions to the model.
2. Parse the chosen tool call.
3. Validate and execute it.
4. Append its result.
5. Ask the model what comes next.
6. Repeat until it produces a normal answer or reaches a step limit.

Some models can request multiple tools in one response, but support varies. Hugging Face's documentation notes that many tool-using models commonly emit one call at a time and that parallel calls introduce additional handling complexity. Build sequential execution first.

Make tool descriptions mutually distinct. `search_documents` should say it searches approved internal text; `lookup_`